

左神算法 树上专题笔记合集 (class117-121)

内容范围: class117 (倍增算法和ST表)、class118 (树上倍增和LCA·上)、class119 (树上倍增和LCA·下)、class120 (树的重心)、class121 (树的直径)。

每道题按四个部分整理: 题面、核心思路、坑点、代码。

class117 倍增算法和ST表

前置知识: 二进制基础 (讲解003)、二分搜索 (讲解006)、辗转相除法求最大公约数 (讲解041)、动态规划基础 (讲解066~069)。

树上倍增与 LCA 在讲解118 开始讲述, 本节是倍增和 ST 表 (稀疏表) 的基础。

一、两个小问题 (位运算基础)

题面

- 给定正数 x , 已知用 m 个二进制位一定能表示 x , 从高位到低位打印 x 每一位的状态 (是 1 还是 0)。
- 给定正数 x , 打印 $\leq x$ 最大的 2 的幂, 是 2 的几次方。

核心思路

- 从第 $m-1$ 位到第 0 位, 用 $1 \ll p$ 去试减: 减得动则这一位是 1 ($t -= 1 \ll p$), 减不动则是 0。本质就是贪心地把 x 从高位向低位分解成二进制。
- 找 $\leq x$ 最大的 2 的幂: 用 `while ((1 << power) <= (x >> 1)) power++;`, 即用「当前 2 的幂是否 $\leq x/2$ 」来判断能否安全翻倍。

坑点

- 问题 2 的朴素写法 `while ((1 << power) <= x) power++; power--;` **没有考虑溢出**: 当 x 逼近 `INT_MAX` 时, `1 << power` 会溢出成负数导致死循环。必须改用 `(1 << power) <= (x >> 1)` 的防溢出写法。

代码

```
1 void show1(int x, int m) {
2     for (int p = m - 1, t = x; p >= 0; p--) {
3         if ((1 << p) <= t) {
4             t -= 1 << p;
5             cout << x << "的第" << p << "位是1\n";
6         } else {
7             cout << x << "的第" << p << "位是0\n";
8         }
9     }
10 }
11
12 void show2(int x) {
13     int power = 0;
```

```

14 // 防止溢出的写法
15 while ((1 << power) <= (x >> 1)) {
16     power++;
17 }
18 cout << "<=" << x << "最大的2的幂，是2的" << power << "次方\n";
19 }

```

二、国旗计划（倍增 + 覆盖环）

题面

给定 m 个点（编号 $1 \sim m$ ）围成一个环， i 号点顺时针到达 $i+1$ 号点， m 号点顺时针回到 1 号点。给定 n 条线段 (a, b) 表示从点 a 顺时针到点 b 。输入保证所有线段能覆盖整个环，且每条线段不会完全在另一条线段内部（可能有重合但互不包含）。返回长度 n 的结果数组 `ans`，`ans[x]` 表示**一定选 x 号线段**的情况下，至少选几条线段能覆盖整个环。

测试链接：<https://www.luogu.com.cn/problem/P4155>

核心思路

- 环拆成数轴**：把环复制成 2 倍长的数轴。若线段左边界 $>$ 右边界（跨越了 $m \rightarrow 1$ ），把右边界 $+= m$ 修正。
- 排序 + 克隆**：按左边界对线段排序，再克隆一份（左右边界各 $+= m$ ），得到 $e = 2n$ 条线段，编号复用原编号。
- 定义"跳一步"**：每条线段跳一步能到达的最右线段 = 右边界以内、左边界最靠右的那条线段（因为线段互不包含，最靠右的开头一定蹦得最远）。用双指针 `arrive`（不回退）一次求出所有 `stjump[i][0]`。
- 倍增建表**：`stjump[i][p] = stjump[stjump[i][p-1]][p-1]`，`power = log2(n)` 决定列数。
- 求答案**：对每条线段 i ，目标 `aim = 左边界 + m`（转一圈回到自己），从高位到低位枚举，只要 `stjump[cur][p]` 的右边界 $<$ `aim` 就跳（累加 $1 < p$ 步），最后 `ans = 跳的步数 + 1 + 1`（+1 是起始线段本身，+1 是最后一步补到覆盖）。

坑点

- 线段 3 个信息需要 `struct Line { id, l, r }`，因为要 `sort`（C 风格 `int[m][3]` 不能直接 `sort`）。
- 先修正「 $l > r$ 」再排序；克隆要在排序之后做，且克隆后天然仍有序。
- 总线段数是 $2n$ ，ST 表列数由 `power = log2(n)` 决定（跳的步数不会超过 n ）。
- `jump` 里的 `next != 0` 判断依赖 `stjump` 全局零初始化，0 表示"空"。

代码

```

1 const int MAXN = 200001;
2 const int LIMIT = 18;
3
4 int power;
5
6 // 每条线段3个信息：线段编号、线段左边界、线段右边界
7 struct Line {
8     int id, l, r;

```

```

9  };
10 Line line[MAXN << 1];
11
12 // stjump[i][p] : 从i号线段出发, 跳的次数是2的p次方, 能到达的最右线段的编号
13 int stjump[MAXN << 1][LIMIT];
14 int ans[MAXN];
15 int n, m;
16
17 // <=n最接近的2的幂, 是2的几次方
18 int log2(int n) {
19     int ans = 0;
20     while ((1 << ans) <= (n >> 1)) ans++;
21     return ans;
22 }
23
24 void build() {
25     for (int i = 1; i <= n; i++)
26         if (line[i].l > line[i].r) line[i].r += m;
27     sort(line + 1, line + n + 1, [](const Line& a, const Line& b) {
28         return a.l < b.l;
29     });
30     for (int i = 1; i <= n; i++) {
31         line[i + n].id = line[i].id;
32         line[i + n].l = line[i].l + m;
33         line[i + n].r = line[i].r + m;
34     }
35     int e = n << 1;
36     for (int i = 1, arrive = 1; i <= e; i++) {
37         while (arrive + 1 <= e && line[arrive + 1].l <= line[i].r) arrive++;
38         stjump[i][0] = arrive;
39     }
40     for (int p = 1; p <= power; p++)
41         for (int i = 1; i <= e; i++)
42             stjump[i][p] = stjump[stjump[i][p - 1]][p - 1];
43 }
44
45 int jump(int i) {
46     int aim = line[i].l + m, cur = i, next, ans = 0;
47     for (int p = power; p >= 0; p--) {
48         next = stjump[cur][p];
49         if (next != 0 && line[next].r < aim) {
50             ans += 1 << p;
51             cur = next;
52         }
53     }
54     return ans + 1 + 1;
55 }

```

三、ST表查询最大值和最小值

题面

给定长度 n 的数组 arr ， m 次查询 $arr[l \sim r]$ 上的最大值和最小值，每次查询打印「最大值 - 最小值」。

测试链接: <https://www.luogu.com.cn/problem/P2880>

核心思路

- 建两张 ST 表 $stmax$ / $stmin$ ， $st[i][p]$ 表示从 i 开始、长度 2^p 区间上的最大值 / 最小值。
- 转移: $st[i][p] = \max(\text{st}[i][p-1], \text{st}[i + 2^{(p-1)}][p-1])$ (左右两半各长 $2^{(p-1)}$)。
- 预处理 $lg[i] = lg[i >> 1] + 1$ ($lg[0] = -1$)， $lg[i]$ 表示 $\leq i$ 最大的 2 的幂是几次方，查询时直接查表得到列号 p 。
- 查询 $[l, r]$: $p = lg[r - l + 1]$ ，结果 = $\max(stmax[l][p], stmax[r - 2^p + 1][p]) - \min(stmin[l][p], stmin[r - 2^p + 1][p])$ 。

坑点

- $LIMIT = 16$ ($2^{15} \leq 50001 < 2^{16}$ ，所以次方 0~15 共 16 列)。
- 查询时左右两段允许重叠——最大/最小值是「可重复贡献问题」，重叠不影响结果。

代码

```
1  const int MAXN = 50001;
2  const int LIMIT = 16;
3
4  int arr[MAXN];
5  int lg[MAXN]; // lg[i] : 查询<=i情况下, 最大的2的幂, 是2的几次方
6  int stmax[MAXN][LIMIT];
7  int stmin[MAXN][LIMIT];
8
9  void build(int n) {
10     lg[0] = -1;
11     for (int i = 1; i <= n; i++) {
12         lg[i] = lg[i >> 1] + 1;
13         stmax[i][0] = arr[i];
14         stmin[i][0] = arr[i];
15     }
16     for (int p = 1; p <= lg[n]; p++)
17         for (int i = 1; i + (1 << p) - 1 <= n; i++) {
18             stmax[i][p] = max(stmax[i][p - 1], stmax[i + (1 << (p - 1))][p - 1]);
19             stmin[i][p] = min(stmin[i][p - 1], stmin[i + (1 << (p - 1))][p - 1]);
20         }
21 }
22
23 int query(int l, int r) {
24     int p = lg[r - l + 1];
25     int a = max(stmax[l][p], stmax[r - (1 << p) + 1][p]);
26     int b = min(stmin[l][p], stmin[r - (1 << p) + 1][p]);
```

```
27     return a - b;
28 }
```

四、ST表查询最大公约数

题面

给定长度 n 的数组 arr ， m 次查询 $arr[l \sim r]$ 上所有数的最大公约数。

测试链接: <https://www.luogu.com.cn/problem/P1890>

核心思路

与最大/最小值 ST 表完全同理，只是合并操作换成 `gcd`（辗转相除法，讲解041）。因为 `gcd` 也是「可重复贡献」运算（左半 `gcd` 与右半 `gcd` 再取 `gcd` 即整体 `gcd`），重叠无影响。

坑点

- `build` 中会调用 `gcd`，而 `gcd` 定义在 `build` 之后，需要**先声明** `int gcd(int a, int b);`。
- `MAXN = 1001`，`LIMIT = 10` ($2^{10} = 1024 > 1001$)。

代码

```
1  const int MAXN = 1001;
2  const int LIMIT = 10;
3
4  int arr[MAXN];
5  int lg[MAXN];
6  int stgcd[MAXN][LIMIT];
7
8  int gcd(int a, int b);
9
10 void build(int n) {
11     lg[0] = -1;
12     for (int i = 1; i <= n; i++) {
13         lg[i] = lg[i >> 1] + 1;
14         stgcd[i][0] = arr[i];
15     }
16     for (int p = 1; p <= lg[n]; p++)
17         for (int i = 1; i + (1 << p) - 1 <= n; i++)
18             stgcd[i][p] = gcd(stgcd[i][p - 1], stgcd[i + (1 << (p - 1))][p - 1]);
19 }
20
21 // 算法讲解041 - 辗转相除法求最大公约数
22 int gcd(int a, int b) {
23     return b == 0 ? a : gcd(b, a % b);
24 }
25
26 int query(int l, int r) {
27     int p = lg[r - l + 1];
28     return gcd(stgcd[l][p], stgcd[r - (1 << p) + 1][p]);
29 }
```

五、出现次数最多的数有几个（有序数组 + 分桶 + ST表）

题面

给定长度 n 的有序数组 arr ， m 次查询 $arr[l \sim r]$ 上出现次数最多的数有几个。

题目查看: <https://www.luogu.com.cn/problem/UVA11235> 提交: <https://vjudge.net/problem/UVA-11235> (对数器验证亦可)

核心思路

利用「有序」这个性质把相同数字归为一个「桶」:

1. $bucket[i]$: 下标 i 属于几号桶; $lft[i]$ / $rgt[i]$: 第 i 号桶的左 / 右边界。桶的词频 = $rgt[i] - lft[i] + 1$ (无需单独存词频数组)。
2. 对「桶的词频」构建最大值 ST 表 $stmax$ 。
3. 查询 $[l, r]$: $lbucket = bucket[l]$, $rbucket = bucket[r]$ 。
 - 若 $lbucket == rbucket$, 答案 = $r - l + 1$ 。
 - 否则: $a = rgt[lbucket] - l + 1$ (左桶命中数), $b = r - lft[rbucket] + 1$ (右桶命中数), 中间桶 ($lbucket+1 \sim rbucket-1$) 若存在则用 ST 表查最大词频 c , 答案 = $\max(a, b, c)$ 。

坑点

- $arr[0]$ 设成极小值 (如 -23333333)，这样从下标 1 开始能自然识别第一个桶的开始。
- $l > r$ 时要先交换 (防「捣乱」查询)。
- 中间桶可能不存在 ($lbucket + 1 == rbucket$ 时)，必须判断 $lbucket + 1 < rbucket$ 才查 ST 表。

代码

```
1  const int MAXN = 100001;
2  const int LIMIT = 17;
3
4  int arr[MAXN];
5  int lg[MAXN];
6  int bucket[MAXN];
7  int lft[MAXN];
8  int rgt[MAXN];
9  int stmax[MAXN][LIMIT];
10
11 void build(int n) {
12     // 题目给定的数值范围-100000 ~ +100000
13     // 把arr[0]设置成不会到达的数字即可
14     arr[0] = -23333333;
15     int cnt = 0;
16     for (int i = 1; i <= n; i++) {
```

```

17         if (arr[i - 1] != arr[i]) {
18             rgt[cnt] = i - 1;
19             lft[++cnt] = i;
20         }
21         bucket[i] = cnt;
22     }
23     rgt[cnt] = n;
24     lg[0] = -1;
25     for (int i = 1; i <= cnt; i++) {
26         lg[i] = lg[i >> 1] + 1;
27         stmax[i][0] = rgt[i] - lft[i] + 1;
28     }
29     for (int p = 1; p <= lg[cnt]; p++)
30         for (int i = 1; i + (1 << p) - 1 <= cnt; i++)
31             stmax[i][p] = max(stmax[i][p - 1], stmax[i + (1 << (p - 1))][p - 1]);
32 }
33
34 int query(int l, int r) {
35     if (l > r) { int tmp = l; l = r; r = tmp; }
36     int lbucket = bucket[l];
37     int rbucket = bucket[r];
38     if (lbucket == rbucket) return r - l + 1;
39     // a : 最左侧桶在此时l~r范围上的数字有几个
40     // b : 最右侧桶在此时l~r范围上的数字有几个
41     int a = rgt[lbucket] - l + 1, b = r - lft[rbucket] + 1, c = 0;
42     if (lbucket + 1 < rbucket) {
43         int from = lbucket + 1, to = rbucket - 1, p = lg[to - from + 1];
44         c = max(stmax[from][p], stmax[to - (1 << p) + 1][p]);
45     }
46     return max(max(a, b), c);
47 }

```

总结：ST 表的适用范围与优缺点

- **适用**：可重复贡献问题（区间最大值、最小值、gcd、区间按位与、区间按位或等）——即两段有重叠也不影响最终答案的查询。
- **不适用**：区间求和等（重叠部分需要额外减掉，不方便）。
- **优点**：构建 $O(n \log n)$ ，构建后单次查询 $O(1)$ ，代码量比线段树小。
- **缺点**：空间 $O(n \log n)$ （比线段树/树状数组大），只能维护可重复贡献信息，且**不支持修改**（一旦改值 ST 表就废了）。

class118 树上问题专题1 - 树上倍增和LCA（上）

前置知识：并查集、链式前向星建图（讲解059）、树形DP（讲解078/079）、倍增基础（讲解117）。
 本节讲求「最低公共祖先（LCA）」的两种方法：**树上倍增**（在线，每次 $O(\log n)$ ）和 **Tarjan**（离线， $O(n+m)$ ）。

一、树节点的第K个祖先

题面

树上有 n 个节点，编号 $0 \sim n-1$ ，用 `parent` 数组给出父节点，根节点是 0 号。实现 `TreeAncestor(n, parent)` 和 `getKthAncestor(i, k)`：返回节点 i 往上跳 k 步的节点，不存在返回 -1 。

测试链接：<https://leetcode.cn/problems/kth-ancestor-of-a-tree-node/>

核心思路

树上倍增最基础的应用：

1. 链式前向星建**有向图**（每个节点只有一个父节点，父 $\rightarrow$ 子建边即可）。
2. dfs 生成 `deep[i]`（节点在第几层）和 `stjump[i][p]`（ i 往上跳 2^p 步到达的节点）。
 - 转移：`stjump[i][p] = stjump[stjump[i][p-1]][p-1]`（只依赖上方的节点，树形DP自顶向下填）。
3. 查询第 k 个祖先：
 - 若 `deep[i] <= k` 返回 -1 （越过了根）。
 - 目标层数 `s = deep[i] - k`，从高位枚举 p ，只要 `deep[stjump[i][p]] >= s` 就往上跳，最后 i 就是答案。

坑点

- `power = log2(n)` 决定 st 表列数（本题 `MAXN=50001`，`LIMIT=16`）。
- 根节点 0 的 `deep[0] = 1`（单独判断），`stjump[0][0] = 0`（指向不存在的节点）。

代码

```
1  const int MAXN = 50001;
2  const int LIMIT = 16;
3  int power;
4  int head[MAXN], nxt[MAXN], to[MAXN], cnt;
5  int deep[MAXN];
6  int stjump[MAXN][LIMIT];
7
8  int log2(int n) {
9      int ans = 0;
10     while ((1 << ans) <= (n >> 1)) ans++;
11     return ans;
12 }
13
14 void addEdge(int u, int v) {
15     nxt[cnt] = head[u]; to[cnt] = v; head[u] = cnt++;
16 }
17
18 // 当前来到i节点，i节点父亲节点是f
19 void dfs(int i, int f) {
20     deep[i] = (i == 0) ? 1 : deep[f] + 1;
```

```

21     stjump[i][0] = f;
22     for (int p = 1; p <= power; p++)
23         stjump[i][p] = stjump[stjump[i][p - 1]][p - 1];
24     for (int e = head[i]; e != 0; e = nxt[e])
25         dfs(to[e], i);
26 }
27
28 class TreeAncestor {
29 public:
30     TreeAncestor(int n, vector<int>& parent) {
31         power = log2(n);
32         cnt = 1;
33         memset(head, 0, n * sizeof(int));
34         for (int i = 1; i < (int)parent.size(); i++)
35             addEdge(parent[i], i);
36         dfs(0, 0);
37     }
38
39     int getKthAncestor(int i, int k) {
40         if (deep[i] <= k) return -1;
41         int s = deep[i] - k; // s是想要去往的层数
42         for (int p = power; p >= 0; p--)
43             if (deep[stjump[i][p]] >= s) i = stjump[i][p];
44         return i;
45     }
46 };

```

二、最近公共祖先（树上倍增法）

题面

给定一棵树（ n 个节点）， m 次查询两个节点的最低公共祖先（LCA）。 n 可达 5×10^5 。

测试链接：<https://www.luogu.com.cn/problem/P3379>

核心思路

1. 建**无向图**（题目给定头节点 root，边是无向的），从 root 开始 dfs 生成 deep 数组和 stjump 表。
2. LCA 查询分两步：
 - **先对齐深度**：让较深的 a 往上跳到与 b 同层（用倍增，保证 `deep[stjump[a][p]] >= deep[b]`）。
 - **再一起往上跳**：从高位枚举 p，只要 `stjump[a][p] != stjump[b][p]` 就一起跳（保证不会跳过 LCA）；最后 `return stjump[a][0]`（再往上一步就是 LCA）。
 - 特判：对齐后若 `a == b`，说明 b 就是 LCA。

坑点

- $n=5 \times 10^5$, $LIMIT=20$ (2^{19} 够用)。树可能退化成一条链, 递归 dfs 在 Java 会爆栈, **C++ 能通过**。
- 单次查询 $O(\log n)$, m 次 $O(m \log n)$ 。倍增是**在线**算法。

代码 (递归版)

```
1  const int MAXN = 500001;
2  const int LIMIT = 20;
3  int power;
4  int head[MAXN], nxt[MAXN << 1], to[MAXN << 1], cnt;
5  int deep[MAXN];
6  int stjump[MAXN][LIMIT];
7
8  void dfs(int u, int f) {
9      deep[u] = deep[f] + 1;
10     stjump[u][0] = f;
11     for (int p = 1; p <= power; p++)
12         stjump[u][p] = stjump[stjump[u][p - 1]][p - 1];
13     for (int e = head[u]; e != 0; e = nxt[e])
14         if (to[e] != f) dfs(to[e], u);
15 }
16
17 int lca(int a, int b) {
18     if (deep[a] < deep[b]) { int tmp = a; a = b; b = tmp; }
19     for (int p = power; p >= 0; p--)
20         if (deep[stjump[a][p]] >= deep[b]) a = stjump[a][p];
21     if (a == b) return a;
22     for (int p = power; p >= 0; p--)
23         if (stjump[a][p] != stjump[b][p]) { a = stjump[a][p]; b = stjump[b][p]; }
24     return stjump[a][0];
25 }
```

三、最近公共祖先 (Tarjan 算法)

题面

同题二, 但换用 Tarjan 离线算法。

核心思路

Tarjan 是**离线**批量算法: 把 m 个查询先挂到每个节点的「问题列表」上 (链式前向星存储), 然后 **dfs 遍历整棵树一遍**, 用并查集维护集合, 就得到所有答案, 总复杂度 $O(n+m)$ 。

过程 (递归版):

1. 来到节点 u , 标记 `visited[u] = true`。
2. 遍历每个孩子 v ($v \neq f$), 递归 `tarjan(v, u)`, 返回后把 v 的集合合并到 u (`father[v] = u`)。
3. 处理 u 的问题列表: 对每个 (u, v) , 若 `visited[v]` 为 true, 则 `ans[查询编号] = find(v)` (v 所在集合的代表节点就是 LCA)。

关键：每遍历完一棵子树，就把整棵子树合并到当前节点，并把当前节点设为代表节点。当查询的两个点中后出现的那个点被访问时，先出现的点所在集合的代表节点恰好就是它们的 LCA。

坑点

- 需要**两套链式前向星**：一套存树的边（headEdge/edgeNext/edgeTo），一套存每个节点的问题列表（headQuery/queryNext/queryTo/queryIndex）。
- 每条查询要**双向**挂到两个节点上（addQuery(u,v,i) 和 addQuery(v,u,i)）。
- 递归版在 Java 会爆栈（find 递归 + tarjan 递归），C++ 能过。
- 迭代版：并查集 find 改成非递归（用栈记录路径再扁平化），tarjan 改成显式栈（u/f/e 三元组）。为便于迭代，先把递归版改成「等义」写法：把 `father[v]=u` 挪到遍历完所有子树之后统一执行 `father[u]=f`。

代码（递归版）

```
1  const int MAXN = 500001;
2  int headEdge[MAXN], edgeNext[MAXN << 1], edgeTo[MAXN << 1], tcnt;
3  int headQuery[MAXN], queryNext[MAXN << 1], queryTo[MAXN << 1], queryIndex[MAXN << 1],
   qcnt;
4  bool visited[MAXN];
5  int father[MAXN];
6  int ans[MAXN];
7
8  int find(int i) {
9      if (i != father[i]) father[i] = find(father[i]);
10     return father[i];
11 }
12
13 void tarjan(int u, int f) {
14     visited[u] = true;
15     for (int e = headEdge[u], v; e != 0; e = edgeNext[e]) {
16         v = edgeTo[e];
17         if (v != f) { tarjan(v, u); father[v] = u; }
18     }
19     for (int e = headQuery[u], v; e != 0; e = queryNext[e]) {
20         v = queryTo[e];
21         if (visited[v]) ans[queryIndex[e]] = find(v);
22     }
23 }
```

总结

方法	复杂度	在线/离线	适用
树上倍增	预处理 $O(n \log n)$ ，单次 $O(\log n)$	在线	高频单点查询
Tarjan	一次遍历 $O(n+m)$	离线（批量）	一次性给出一批查询

递归改迭代（显式栈）通用套路：栈里存 (u, f, e) 三元组—— u 当前节点、 f 父节点、 e 处理到几号边。 $e == -1$ 表示第一次到达（做“进入节点”的工作）， $e == 0$ 表示所有边处理完（做“离开节点”的工作）；压栈时先把自己 $(u, f, \text{下一条边})$ 压回，再压孩子 $(child, u, -1)$ 。当数据量很大、递归可能爆栈时，必须掌握。

class119 树上问题专题2 - 树上倍增和LCA（下）

前置知识：讲解118（树上倍增与LCA）、讲解061（最小生成树 Kruskal）、讲解105（字符串哈希）。

本节用树上倍增 + LCA 解决 5 道综合题。核心结论：**树上两点距离** $dist(a,b) = deep[a] + deep[b] - 2*deep[lca]$ 。

一、紧急集合

题面

树上有 n 个节点，每条边权为 1。每次查询 (a,b,c) ：三个人分别站在 a 、 b 、 c ，选一个点集合，求三人走过的总边数最少是多少，输出集合点和最小花费。 $n, m \leq 5 \times 10^5$ 。

测试链接：<https://www.luogu.com.cn/problem/P4281>

核心思路

对三点两两求 LCA，得到 $h1=lca(a,b)$ 、 $h2=lca(a,c)$ 、 $h3=lca(b,c)$ 。关键结论：**这三个 LCA 只有 1 个或 2 个不同的点**。

- 取 $high$ = 较浅（层数小）的那个 LCA， low = 较深（层数大）的那个（或唯一的那个）。
- 汇聚点 = low （三人往较低的那个祖先汇聚最省）。
- 花费 = $deep[a] + deep[b] + deep[c] - 2*deep[high] - deep[low]$ （两条边权为 1，所以距离 = 层数差）。

推导：三人先各到 low （此时 a 、 b 已汇合在 $high$ 以下）， c 额外走到 low ，因此总距离 = $(deep[a]-deep[high]) + (deep[b]-deep[high]) + (deep[c]-deep[low])$ ，展开即上式。

坑点

- 用 `long long` 存 `cost`（虽然本题数值不溢出，但题目 n 大，谨慎）。
- 洛谷对 Java 空间卡得严，只有 C++ 能全过（逻辑两个语言完全一样）。

代码

```
1 void compute(int a, int b, int c) {
2     // 来自对结构关系的深入分析
3     int h1 = lca(a, b), h2 = lca(a, c), h3 = lca(b, c);
4     int high = h1 != h2 ? (deep[h1] < deep[h2] ? h1 : h2) : h1;
5     int low = h1 != h2 ? (deep[h1] > deep[h2] ? h1 : h2) : h3;
6     together = low;
7     cost = (long long)deep[a] + deep[b] + deep[c] - (long long)deep[high] * 2 -
8     deep[low];
9 }
```

二、货车运输

题面

n 座城市， m 条双向道路 (u,v,w) 限重 w 。汽车载重不能超过路径上任意一条边的限重。每次查询 (a,b) ：从 a 到 b 的最大载重是多少（即路径上最小边权的最大值）；不连通返回 -1 。 $n \leq 10^4$, $m \leq 5 \times 10^4$, $q \leq 3 \times 10^4$ 。

测试链接：<https://www.luogu.com.cn/problem/P1967>

核心思路

- 最大生成树**：先收集所有边，Kruskal 按权值从大到小排序（贪心：能走大边就不走小边），用并查集选边，选中的边建一棵树（图可能不连通 $\rightarrow$ 森林）。
- 倍增 + 路径最小值**：除 `stjump[u][p]` 外再维护 `stmin[u][p]`（ u 往上跳 2^p 步的路径中最小的边权）。
 - 转移：`stmin[u][p] = min(stmin[u][p-1], stmin[stjump[u][p-1]][p-1])`。
- 查询**：先 `find(a) != find(b)` 判断是否连通（ -1 ）。然后像求 LCA 一样对齐深度、一起往上跳，过程中用 `ans = min(ans, stmin[...])` 维护路径最小边权。注意这里是求**最小边权**而非 LCA 节点，所以函数名叫 `lca` 但返回的是 `min`。

坑点

- 边需要 `struct Edge { u, v, w; }` 并排序（C 风格 `int[m][3]` 不能 `sort`）。
- 可能是森林，要 `for i in 1..n: if(!visited[i]) dfs(i,0,0)`，每个连通块各自 `dfs`。
- 每个树的根 ($f=0$)：`stjump[u][0]=u`, `stmin[u][0]=INT_MAX`（根上面没有边）。

代码

```
1 void kruskal(int n, int m) {
2     sort(edges + 1, edges + m + 1, [](const Edge& a, const Edge& b) { return a.w > b.w; });
3     for (int i = 1, a, b, fa, fb; i <= m; i++) {
4         a = edges[i].u; b = edges[i].v;
5         fa = find(a); fb = find(b);
6         if (fa != fb) { father[fa] = fb; addEdge(a, b, edges[i].w); addEdge(b, a,
7         edges[i].w); }
```

```

7     }
8 }
9
10 void dfs(int u, int w, int f) {
11     visited[u] = true;
12     if (f == 0) { deep[u] = 1; stjump[u][0] = u; stmin[u][0] = INT_MAX; }
13     else { deep[u] = deep[f] + 1; stjump[u][0] = f; stmin[u][0] = w; }
14     for (int p = 1; p <= power; p++) {
15         stjump[u][p] = stjump[stjump[u][p - 1]][p - 1];
16         stmin[u][p] = min(stmin[u][p - 1], stmin[stjump[u][p - 1]][p - 1]);
17     }
18     for (int e = head[u]; e != 0; e = nxt[e])
19         if (!visited[to[e]]) dfs(to[e], weight[e], u);
20 }

```

三、边权相等的最小修改次数

题面

一棵树 ($0 \sim n-1$)，每条边有权重 w ($1 \leq w \leq 26$)。每次查询 (a, b) ：把 $a \rightarrow b$ 路径上所有边变成同一种值，至少需要修改几条边。

测试链接：<https://leetcode.cn/problems/minimum-edge-weight-equilibrium-queries-in-a-tree/>

核心思路

利用「权值只有 1~26 种」这个关键约束：

1. **dfs 统计前缀词频**：`weightCnt[u][w]` = 从根到 u 的路径上，权值为 w 的边有几条。转移：先复制父亲的值，再把当前边权 `weightCnt[u][w]++`。
2. **tarjan 离线求所有查询的 LCA**，存到 `lca` 数组。
3. **查询**：对 $w=1..26$ 枚举，`wcnt = weightCnt[a][w] + weightCnt[b][w] - 2*weightCnt[lca][w]`（路径上权值为 w 的边数）。累加得 `allCnt`（总边数），取最大 `maxCnt`，答案 = `allCnt - maxCnt`。

坑点

- `weightCnt` 是二维数组 `[MAXN][MAXW+1]`（下标 $0 \sim 26$ ，0 恒为 0）。
- tarjan 里 `father[u] = f` 放在遍历完子树之后（等义写法）。
- 权值种类少（26），所以单次查询 $O(26)$ ，整体线性。

代码

```

1 void dfs(int u, int w, int f) {
2     if (u == 0) memset(weightCnt[u], 0, sizeof(weightCnt[u]));
3     else {
4         for (int i = 1; i <= MAXW; i++) weightCnt[u][i] = weightCnt[f][i];
5         weightCnt[u][w]++;
6     }
7     for (int e = headEdge[u]; e != 0; e = edgeNext[e])

```

```

8         if (edgeTo[e] != f) dfs(edgeTo[e], edgevalue[e], u);
9     }
10
11 void tarjan(int u, int f) {
12     visited[u] = true;
13     for (int e = headEdge[u]; e != 0; e = edgeNext[e])
14         if (edgeTo[e] != f) tarjan(edgeTo[e], u);
15     for (int e = headQuery[u], v; e != 0; e = queryNext[e]) {
16         v = queryTo[e];
17         if (visited[v]) lca[queryIndex[e]] = find(v);
18     }
19     father[u] = f;
20 }

```

四、在传球游戏中最大化函数值

题面

n 名玩家 ($0 \sim n-1$) , `receiver[i]` 表示 i 传球的下一人。从 x 出发传球恰好 k 次, $f(x)$ = 沿途所有触球玩家编号之和 (可重复累加) 。求 $\max f(x)$ 。 k 可达 `long` 量级。

测试链接: <https://leetcode.cn/problems/maximize-value-of-function-in-a-ball-passing-game/>

核心思路

倍增 + 前缀和 (本质是基环树, 但倍增法最好想) :

1. `stjump[i][p]` : i 跳 2^p 步到达的玩家; `stsum[i][p]` : i 跳 2^p 步沿途编号之和。
 - 转移: `stjump[i][p] = stjump[stjump[i][p-1]][p-1]`, `stsum[i][p] = stsum[i][p-1] + stsum[stjump[i][p-1]][p-1]`。
2. 把 k 二进制分解 (`kbits` 存 k 的哪些位是 1) , 对每个起点 i , `sum = i` , 按 `kbits` 逐块累加并跳转, 取最大值。

坑点

- k 是 `long`, 用 `1LL << power` 防溢出, `stsum` 用 `long long`。
- `LIMIT = 34` (k 最多约 2^{34}) 。
- 最优解是基环树 DP ($O(n)$) , 倍增法 $O(n \log n)$ 也能过但常数较大。

代码

```

1 void build(long long k) {
2     power = 0;
3     while ((1LL << power) <= (k >> 1)) power++;
4     m = 0;
5     for (int p = power; p >= 0; p--)
6         if ((1LL << p) <= k) { kbites[m++] = p; k -= 1LL << p; }
7 }
8

```

```

9  long long getMaxFunctionValue(vector<int>& receiver, long long k) {
10     build(k);
11     int n = receiver.size();
12     for (int i = 0; i < n; i++) { stjump[i][0] = receiver[i]; stsum[i][0] =
receiver[i]; }
13     for (int p = 1; p <= power; p++)
14         for (int i = 0; i < n; i++) {
15             stjump[i][p] = stjump[stjump[i][p - 1]][p - 1];
16             stsum[i][p] = stsum[i][p - 1] + stsum[stjump[i][p - 1]][p - 1];
17         }
18     long long sum, ans = 0;
19     for (int i = 0, cur; i < n; i++) {
20         cur = i; sum = i;
21         for (int j = 0; j < m; j++) { sum += stsum[cur][kbits[j]]; cur = stjump[cur]
[kbits[j]]; }
22         ans = max(ans, sum);
23     }
24     return ans;
25 }

```

五、检查树上两节点间的路径是否是回文

题面

一棵树 ($1 \sim n$, 根为 1), 每个节点有一个小写字母。每次查询 (a,b): 判断 $a \rightarrow b$ 路径上节点字符组成的字符串是否是回文。 $n, m \leq 10^5$ 。

测试链接: <https://ac.nowcoder.com/acm/contest/78807/G>

核心思路

用字符串哈希把路径字符串变成数值, 倍增快速求哈希值。

1. 字符转数字: $a \rightarrow 1, \dots, z \rightarrow 26$ 。取进制 $K=499$, 预处理 $kpow[i] = K^i$ 。

2. 建两张哈希表:

- $stup[u][p]$: u 往上走 2^p 步的路径哈希 (不含 u 本身, 即从父节点往上)。 $stup[u][0] = s[f]$ 。
- $stdown[u][p]$: 从某祖先往下走 2^p 步到达 u 的路径哈希 (不含 u 本身)。 $stdown[u][0] = s[f]$ 。
- 转移 (设 $v = jump[u][p-1]$, 步长 $len = 2^{(p-1)}$):

$stup[u][p] = stup[u][p-1] * kpow[len] + stup[v][p-1]$
 $stdown[u][p] = stdown[v][p-1] * kpow[len] + stdown[u][p-1]$

3. $hash(from, anc, to)$: 上坡 ($from \rightarrow anc$, 用 $stup$, up 初值 = $s[from]$) + 下坡 (anc 下一节点 $\rightarrow to$, 用 $stdown$, $down$ 初值 = $s[to]$), 拼成 $up * kpow[height] + down$ 。

4. 判断回文: $hash(a, lca, b) == hash(b, lca, a)$ (正序哈希 == 逆序哈希)。

坑点

- 哈希是「自然溢出」——Java 的 long 与 C++ 的 `long long` 都靠溢出截断，比较的是相等性，两个方向按同样规则溢出即可。
- 上坡跳的判定是 `deep[jump[from][p]] >= deep[lca]`（能到 lca），下坡是 `deep[jump[to][p]] > deep[lca]`（严格大于，不能到 lca）。
- 函数名 `hash` 与 `std::hash` 冲突，需改名（如 `calHash`）；局部变量名 `lca` 与函数 `lca` 冲突，需改名（如 `anc`）。

代码

```
1 void dfs(int u, int f) {
2     deep[u] = deep[f] + 1;
3     jump[u][0] = f;
4     stup[u][0] = stdown[u][0] = s[f];
5     for (int p = 1, v; p <= power; p++) {
6         v = jump[u][p - 1];
7         jump[u][p] = jump[v][p - 1];
8         stup[u][p] = stup[u][p - 1] * kpow[1 << (p - 1)] + stup[v][p - 1];
9         stdown[u][p] = stdown[v][p - 1] * kpow[1 << (p - 1)] + stdown[u][p - 1];
10    }
11    for (int e = head[u], v; e != 0; e = nxt[e]) {
12        v = to[e];
13        if (v != f) dfs(v, u);
14    }
15 }
16
17 long long calHash(int from, int anc, int to) {
18     long long up = s[from];
19     for (int p = power; p >= 0; p--)
20         if (deep[jump[from][p]] >= deep[anc]) { up = up * kpow[1 << p] + stup[from][p];
from = jump[from][p]; }
21     if (to == anc) return up;
22     long long down = s[to];
23     int height = 1;
24     for (int p = power; p >= 0; p--)
25         if (deep[jump[to][p]] > deep[anc]) { down = stdown[to][p] * kpow[height] +
down; height += 1 << p; to = jump[to][p]; }
26     return up * kpow[height] + down;
27 }
28
29 bool isPalindrome(int a, int b) {
30     int anc = lca(a, b);
31     return calHash(a, anc, b) == calHash(b, anc, a);
32 }
```

总结

本节 5 道题都是「树上倍增/LCA 模板 + 特定改写」：

题目	结合的知识点	关键改写
紧急集合	距离公式	三点 LCA 的 high/low 分析
货车运输	最大生成树	stmin 维护路径最小边权
边权修改	tarjan + 前缀词频	权值只有 26 种，枚举
传球游戏	倍增	stsum 维护编号和，二进制分解 k
路径回文	字符串哈希	stup/stdown 正逆序哈希

难点不在倍增/LCA 本身，而在把不同知识点与树结构结合起来的分析。

class120 树的重心

树上问题专题三。前置：链式前向星建图（讲解059）、树形DP（讲解078/079）、递归改迭代（讲解118）。

树的重心的定义与性质

三种定义（虽然定义不同，但求出来的重心是同一个，或同两个）：

1. 以某个节点为根时，**最大子树的节点数最少**，该节点是重心；
2. 以某个节点为根时，**每棵子树的节点数都不超过总节点数的一半**，该节点是重心；
3. **所有节点走向该节点的总距离和最小**，该节点是重心。

补充性质：

4. 一棵树**最多有两个重心**；若有两个，一定通过一条边相邻；
5. 增加/删除一个**叶节点**后，重心要么不动，要么最多移动一条边；
6. 两棵树连起来后，新树的重心一定在原来两棵树重心的**路径**上；
7. 边权都为正时，所有节点汇聚到**重心**走过的总距离最小（与边权怎么分布无关）。

题目一：平衡行为（POJ 1655）— 重心求解方式一

题面

n 个节点、 $n-1$ 条边的树。返回重心点编号，以及该重心最大子树的节点数；若重心不唯一，返回编号较小的那个。

核心思路

用第一种定义求解。从 1 号点开始 DFS，收集每个节点 u 的 $size[u]$ （以 1 为根时 u 这棵子树的节点数）。以 u 为根时，它分成的"子树"包括：

- 每个孩子 v 对应的子树 $size[v]$ ；
- 加上"上面那一块"： $n - size[u]$ 。

取这些值的最大值 $maxsub$ ，就是 u 做根时最大子树的节点数。全局维护 $maxsub$ 最小的节点（并列取编号更小者）即为重心。

坑点

- 别忘了"上面那一块" `n - size[u]`，很多人漏掉。
- 重心可能有两个，题目要求返回编号较小者，需要 `maxsub < best || (maxsub == best && u < center)` 的判断。

代码

```

1  const int MAXN = 20001;
2  int n, cnt;
3  int head[MAXN], nxt[MAXN << 1], to[MAXN << 1];
4  int size[MAXN], center, best;
5
6  void dfs(int u, int f) {
7      size[u] = 1;
8      // 以当前节点u做根节点，最大的子树有多少节点
9      int maxsub = 0;
10     for (int e = head[u], v; e != 0; e = nxt[e]) {
11         v = to[e];
12         if (v != f) {
13             dfs(v, u);
14             size[u] += size[v];
15             maxsub = max(maxsub, size[v]);
16         }
17     }
18     maxsub = max(maxsub, n - size[u]);
19     // 题目要求找到编号最小的重心
20     if (maxsub < best || (maxsub == best && u < center)) {
21         best = maxsub;
22         center = u;
23     }
24 }
```

题目二：教父 (POJ 3107) — 重心求解方式二

题面

n 个节点、 $n-1$ 条边的树。返回**所有**重心点（最多两个）；一个重心打印一个数，两个重心打印 `a b`（空格分隔）。

核心思路

用第二种定义求解。DFS 收集每个节点的 `size[u]` 和 `maxsub[u]`（以 u 为根时最大子树的节点数，同样 = 孩子子树最大值与 `n - size[u]` 的较大者）。最后统一扫一遍：`maxsub[i] <= n / 2` 的节点就是重心。因为重心最多两个，用一个长度为 2 的数组收集即可。

坑点

- 和方式一一样，`maxsub[u]` 的"上面那块"是 `n - size[u]`，不能漏。
- 方式二不需要全局 `best/center`，每个节点各自存 `maxsub`，扫一遍判定即可，思路更直接。

代码

```

1  const int MAXN = 50001;
2  int n, cnt;
3  int head[MAXN], nxt[MAXN << 1], to[MAXN << 1];
```

```

4  int size[MAXN], maxsub[MAXN];
5
6  void dfs(int u, int f) {
7      size[u] = 1;
8      maxsub[u] = 0;
9      for (int e = head[u], v; e != 0; e = nxt[e]) {
10         v = to[e];
11         if (v != f) {
12             dfs(v, u);
13             size[u] += size[v];
14             maxsub[u] = max(maxsub[u], size[v]);
15         }
16     }
17     maxsub[u] = max(maxsub[u], n - size[u]);
18 }
19 // 主逻辑: for i in [1,n], 若 maxsub[i] <= n/2 则收集进 centers[]。

```

题目三：牛群聚集（P2986）——带点权的重心

题面

n 个农场（节点），每个农场有若干头牛（点权）， $n-1$ 条带权值的双向边（边权）。所有牛要汇聚到同一个点，求走过的**总距离**最小是多少（返回总距离，不需要返回汇聚点）。

核心思路

边权都是正数，由性质 7：所有牛汇聚到**重心**时总距离最小。所以两步：

1. **求带点权的重心**：把 `size` 的含义从"节点数"换成"牛的总量"——
 - `size[u]` 初始值不再是 1，而是 `cow[u]`（本农场牛数）；
 - 最后"上面那块" = `cowSum - size[u]`（牛的总数减去 u 子树牛量）。
 其余与普通重心求法一致（用第一种定义，维护最大子树牛量最小的点）。
2. **算总距离**：找到重心 `center` 后，从 `center` 出发 DFS 得到每个点 `path[i]`（到重心的距离），答案 = $\sum cow[i] * path[i]$ 。

坑点

- 递归版会**爆栈**（ n 可达 10^5 ），必须改成迭代版（自己压栈模拟递归，见讲解118）。`findCenter` 里"先 DFS 收集完子树，再统一统计"的写法就是为了方便改迭代。
- `size` 初始值 = `cow[u]`，不是 1；"上面那块"是 `cowSum - size[u]`，不是 `n - size[u]`。
- 答案要用 `long long`（牛数 $\times$ 距离可能溢出 int）。

代码（递归版核心）

```

1  const int MAXN = 100001;
2  int n, cnt, cowSum, best, center;
3  int cow[MAXN], head[MAXN], nxt[MAXN << 1], to[MAXN << 1], weight[MAXN << 1];
4  int size[MAXN];    // size[i] : 以i为头的子树，牛的总量
5  int path[MAXN];    // path[i] : 从重心到达i的距离
6
7  void findCenter(int u, int f) {
8      size[u] = cow[u];

```

```

9   for (int e = head[u], v; e != 0; e = nxt[e]) {
10       v = to[e];
11       if (v != f) findCenter(v, u);
12   }
13   // 遍历完成后再统一统计（为了后续改迭代版方便）
14   int maxsub = 0;
15   for (int e = head[u], v; e != 0; e = nxt[e]) {
16       v = to[e];
17       if (v != f) {
18           size[u] += size[v];
19           maxsub = max(maxsub, size[v]);
20       }
21   }
22   maxsub = max(maxsub, cowSum - size[u]);
23   if (maxsub < best) { best = maxsub; center = u; }
24 }
25
26 void setPath(int u, int f) {
27     for (int e = head[u], v; e != 0; e = nxt[e]) {
28         v = to[e];
29         if (v != f) { path[v] = path[u] + weight[e]; setPath(v, u); }
30     }
31 }
32
33 long long compute() {
34     for (int i = 1; i <= n; i++) cowSum += cow[i];
35     findCenter(1, 0);
36     path[center] = 0;
37     setPath(center, 0);
38     long long ans = 0;
39     for (int i = 1; i <= n; i++) ans += (long long)cow[i] * path[i];
40     return ans;
41 }

```

迭代版：用 `int ufe[MAXN][3]`（存 `u`、`f`、`e`）作为手写栈，`push/pop` 模拟递归的进入与回溯，逻辑与递归版完全一致。

题目四：删增边使其重心唯一（CF1406C）

题面

n 个节点、 $n-1$ 条边的树。允许先删一条边、再增加一条边，使重心唯一。打印任意一种方案（两行：删的边 `u v`、加的边 `x y`）。

核心思路

先 DFS 求出每个节点最大子树大小 `maxsub`，统计重心数量：

- **只有一个重心**：删掉连接重心的任意一条边，再把这条边原样加回（打印同一条边两次即可）。
- **有两个重心**（设为 `centers[0]`、`centers[1]`）：选定 `centers[0]` 作为唯一重心；在另一个重心 `centers[1]` 里随便找一个叶子，删掉叶子与其父亲的边，把叶子挂到 `centers[0]` 下。

坑点

- 两个重心一定相邻；找叶子时把 `centers[0]` 当作 `centers[1]` 的父亲，从 `centers[1]` 一路往下找，**找到一个叶子就 `return`**，不用遍历整棵树。
- 只有一个重心时打印同一条边两次，这是题目（已实测）的要求。

代码

```

1  const int MAXN = 100001;
2  int n, cnt;
3  int head[MAXN], nxt[MAXN << 1], to[MAXN << 1];
4  int size[MAXN], maxsub[MAXN];
5  int centers[2], leaf, leafFather;
6
7  void dfs(int u, int f) {
8      size[u] = 1; maxsub[u] = 0;
9      for (int e = head[u], v; e != 0; e = nxt[e]) {
10         v = to[e];
11         if (v != f) {
12             dfs(v, u);
13             size[u] += size[v];
14             maxsub[u] = max(maxsub[u], size[v]);
15         }
16     }
17     maxsub[u] = max(maxsub[u], n - size[u]);
18 }
19
20 // 随意找一个叶节点和它的父亲（哪一组都行）
21 void find(int u, int f) {
22     for (int e = head[u]; e != 0; e = nxt[e]) {
23         if (to[e] != f) { find(to[e], u); return; }
24     }
25     leaf = u; leafFather = f;
26 }
27
28 int centerCnt() {
29     int m = 0;
30     for (int i = 1; i <= n; i++)
31         if (maxsub[i] <= n / 2) centers[m++] = i;
32     return m;
33 }
34 // 主逻辑：cnt==1 打印 centers[0] 与其一条邻边两次；
35 //          cnt==2 时 find(centers[1], centers[0])，打印 (leafFather, leaf) 与
    (centers[0], leaf)。

```

class121 树的直径

树上问题专题四。前置：链式前向星建图（059）、树形DP（078/079）、递归改迭代（118）、并查集与集合打标签（056/057）、单调队列（054）。

树的直径：定义、两种求法、两个结论

定义：树上距离最远的两个点形成的路径长度。

两种求法：

	两次 DFS	树形 DP
适用范围	边权非负	所有树（含负边）
遍历次数	两遍	一遍
能得到路径/端点信息	能（last 数组）	只能得到长度

两个结论（边权都为正时成立）：

- 1. 树上有多条直径时，它们一定**共用中间的公共部分**（一个公共点或一段公共路径）；
- 2. 树上任意一点，离它最远的点里，**直径的两个端点至少有一个**在其中。

题目一：树的直径模版（U81904，边权可能为负）

题面

给定一棵树，边权可能为负，求直径长度。

写法 1：两遍 DFS

核心思路

第一次 DFS 从任意点（1 号）出发，找离它最远的点 `start`；第二次 DFS 从 `start` 出发，找离它最远的点 `end`，直径 = `dist[end]`。DFS 过程中顺手维护 `last[i]`（从当前头出发，i 的上一个节点），就能反推出整条直径路径。

坑点

- **只适用于边权非负。**有负边时两次 DFS 会求错（第一次找到的最远点不是直径端点），本题会有用例过不了。
- 好处是能得到路径上所有点，后面的题（公共部分、巡逻、消防）都要用到。

```
1  const int MAXN = 500001;
2  int n, cnt, start, fin, diameter;
3  int head[MAXN], nxt[MAXN << 1], to[MAXN << 1], weight[MAXN << 1];
4  int dist[MAXN], last[MAXN];
5
6  void dfs(int u, int f, int w) {
7      last[u] = f;
8      dist[u] = dist[f] + w;
9      for (int e = head[u]; e != 0; e = nxt[e])
10         if (to[e] != f) dfs(to[e], u, weight[e]);
11 }
12
13 void read() {
14     dfs(1, 0, 0);
15     start = 1;
16     for (int i = 2; i <= n; i++) if (dist[i] > dist[start]) start = i;
```

```

17     dfs(start, 0, 0);
18     fin = 1;
19     for (int i = 2; i <= n; i++) if (dist[i] > dist[fin]) fin = i;
20     diameter = dist[fin];
21 }

```

写法 2: 树形 DP

核心思路

每个节点维护两个信息:

- `dist[u]`: 从 `u` **必须往下走** (可不选任何边) 能走出的最大距离, 初始为 0;
- `ans[u]`: 路径**必须包含 u 且以 u 为最高点**时 (最多往两条支路走) 的最大路径长度, 初始为 0。

孩子信息收集完后, **先更新 ans 再更新 dist** (保证"两条支路"拼起来时, `dist[u]` 还代表"之前某一条支路"的最优值):

```

1  ans[u] = max(ans[u], dist[u] + dist[v] + weight[e]);
2  dist[u] = max(dist[u], dist[v] + weight[e]);

```

最终答案 = 所有 `ans[i]` 的最大值。

坑点

- `dist` 初始必须是 0 (可以"一条边都不选", 自己单独待着, 尤其有负边时)。
- 顺序不能反: 先 `ans` 后 `dist`。

```

1  int dist[MAXN], ans[MAXN];
2
3  void dp(int u, int f) {
4      for (int e = head[u], v; e != 0; e = nxt[e]) {
5          v = to[e];
6          if (v != f) dp(v, u);
7      }
8      for (int e = head[u], v; e != 0; e = nxt[e]) {
9          v = to[e];
10         if (v != f) {
11             ans[u] = max(ans[u], dist[u] + dist[v] + weight[e]);
12             dist[u] = max(dist[u], dist[v] + weight[e]);
13         }
14     }
15 }
16 // 答案 = max(ans[1..n])

```

写法 3: 树形 DP 化简

`ans` 数组可以省掉: DP 过程中需要决策 `ans[u]` 的地方, 直接去更新一个全局 `diameter`。

```

1  int dist[MAXN], diameter;    // diameter 初始 INT_MIN
2
3  void dp(int u, int f) {

```

```

4     for (int e = head[u], v; e != 0; e = nxt[e]) {
5         v = to[e];
6         if (v != f) dp(v, u);
7     }
8     for (int e = head[u], v; e != 0; e = nxt[e]) {
9         v = to[e];
10        if (v != f) {
11            diameter = max(diameter, dist[u] + dist[v] + weight[e]);
12            dist[u] = max(dist[u], dist[v] + weight[e]);
13        }
14    }
15 }

```

题目二：所有直径的公共部分 (P3304)

题面

给定一棵边权都为正的树，打印两行：直径长度、所有直径的公共部分有几条边。

核心思路

1. 两次 DFS 得到一条直径 `start` $\rightarrow$ `fin` 以及 `last` (可从 `fin` 跳回 `start`)、`dist` (从 `start` 出发的距离)。
2. 对直径上每个点 `i`，求 `maxDistanceExceptDiameter(i)`：**不经过直径路径上任何点**能走出的最大距离（即该点挂着的“侧枝”最远能走多远）。
3. 从 `fin` 往 `start` 跳，用结论判断公共部分的左右端点：
 - 若 `maxDist == dist[i]` (`i` 到 `start` 的距离)，满足条件；**最晚**满足的点是公共部分最右点 `r`。
 - 若 `maxDist == diameter - dist[i]` (`i` 到 `fin` 的距离)，满足条件；**最早**满足的点是公共部分最左点 `l`。
4. `l == r` 说明所有直径只共用一个点，公共边数为 0；否则从 `l` 沿 `last` 跳到 `r` 数边数。

坑点

- `maxDistanceExceptDiameter` 里两个限制：不能走向 `diameterPath[v] == true` 的节点，也不能走回头路 (`v != f`)。
- 求 `maxDist` 时以 `long long` 存储 (`diameter - dist[i]` 可能较大)。
- 递归版会爆栈，需改迭代版 (见讲解118)。

```

1  const int MAXN = 200001;
2  int n, cnt, start, fin;
3  int head[MAXN], nxt[MAXN << 1], to[MAXN << 1], weight[MAXN << 1];
4  long long dist[MAXN], diameter;
5  int last[MAXN], commonEdges;
6  bool diameterPath[MAXN];
7
8  long long maxDistanceExceptDiameter(int u, int f, long long c) {
9      long long ans = c;
10     for (int e = head[u], v; e != 0; e = nxt[e]) {
11         v = to[e];
12         if (!diameterPath[v] && v != f)

```

```

13         ans = max(ans, maxDistanceExceptDiameter(v, u, c + weight[e]));
14     }
15     return ans;
16 }
17
18 void compute() {
19     road(); // 两次DFS得到 start/fin/diameter
20     for (int i = fin; i != 0; i = last[i]) diameterPath[i] = true;
21     int l = start, r = fin;
22     long long maxDist;
23     for (int i = last[fin]; i != start; i = last[i]) {
24         maxDist = maxDistanceExceptDiameter(i, 0, 0);
25         if (maxDist == diameter - dist[i]) r = i; // 最右
26         if (maxDist == dist[i] && l == start) l = i; // 最左
27     }
28     if (l == r) commonEdges = 0;
29     else {
30         commonEdges = 1;
31         for (int i = last[r]; i != l; i = last[i]) commonEdges++;
32     }
33 }

```

题目三：造公园 (P2195)

题面

n 个节点、 m 条边（边权都是 1），构成一个森林（多个连通区，每个连通区是树）。 q 次操作：

- 1 x ：返回 x 所在连通区的直径长度；
- 2 $x\ y$ ：若已连通则忽略；否则把两块连通区连通，使合并后的大区域直径最小（不要求输出方案）。

核心思路

- **并查集 + 集合打标签**：每个连通区用代表节点存直径 `diameter[代表]`。
- 建图时对每条边 u 、 v 做 `father[find(u)] = find(v)`；然后对每个“根代表”（`i == father[i]`）跑一遍**树形 DP**求该连通区的直径（边权 1）。
- **合并公式**：两区域直径分别为 a 、 b ，新直径 = $\max((a+1)/2 + (b+1)/2 + 1, a, b)$ 。前一项是“卡着两个直径中点连一条边”形成的新直径，后两项是各自旧直径可能更大。

坑点

- 树形 DP 求直径只求长度即可，用 `dist[u] + dist[v] + 1` 决策。
- 合并时注意运算符顺序：`(diameter[x]+1)/2 + (diameter[y]+1)/2 + 1` 是整数除法向上取整。
- `diameter` 数组只在**代表节点**上有意义，非代表节点的值无用。

```

1  const int MAXN = 300001;
2  int cnt;
3  int head[MAXN], nxt[MAXN << 1], to[MAXN << 1];
4  int father[MAXN]; // 并查集
5  int dist[MAXN]; // 树形DP: 从u往下走最大距离
6  int diameter[MAXN]; // 代表节点才有效
7

```

```

8  int find(int i) { return i != father[i] ? father[i] = find(father[i]) : father[i]; }
9
10 void dp(int u, int f) {
11     for (int e = head[u], v; e != 0; e = nxt[e]) {
12         v = to[e];
13         if (v != f) dp(v, u);
14     }
15     for (int e = head[u], v; e != 0; e = nxt[e]) {
16         v = to[e];
17         if (v != f) {
18             diameter[u] = max(diameter[u], max(diameter[v], dist[u] + dist[v] + 1));
19             dist[u] = max(dist[u], dist[v] + 1);
20         }
21     }
22 }
23 // 合并: father[x]=y; diameter[y]=max((diameter[x]+1)/2+(diameter[y]+1)/2+1,
    max(diameter[x],diameter[y]));

```

题目四：巡逻 (P3629)

题面

n 个节点、 $n-1$ 条边（边权都是 1）的树，警车从 1 号点出发走遍所有边再回到 1 号点。可新建 k （1 或 2）条边，每条新建道路必须**恰好经过一次**，使经过的边数最少，返回最小值。

核心思路

- 不新建道路：每条边走两次，共 $2*(n-1)$ 。
- $k=1$ ：新建一条边连起直径两端，直径上的边都少走一次，答案 = $2*(n-1) - \text{diameter1} + 1$ 。
- $k=2$ ：先求直径 `diameter1`，把直径上所有点标记 `diameterPath`；把**直径上的边权改成 -1**（其余仍为 1），再用**树形 DP** 求第二遍直径 `diameter2`（负边只能用树形 DP）。答案 = $2*(n-1) - \text{diameter1} - \text{diameter2} + 2$ 。这样公共边的影响被自动抵消掉。

坑点

- 第二次求直径有负边，不能用两遍 DFS，必须树形 DP。
- 边权改成 -1 的方法：DP 里 `w = (diameterPath[u] && diameterPath[v]) ? -1 : 1`（两个端点都在直径上才算直径边）。
- $k=1$ 时无需跑第二遍。

```

1  int n, k, start, fin, diameter1, diameter2;
2  int head[MAXN], nxt[MAXN << 1], to[MAXN << 1];
3  int dist[MAXN], last[MAXN];
4  bool diameterPath[MAXN];
5
6  void dp(int u, int f) { // 树形DP第二次求直径
7      for (int e = head[u], v; e != 0; e = nxt[e]) {
8          v = to[e];
9          if (v != f) dp(v, u);
10     }
11     for (int e = head[u], v, w; e != 0; e = nxt[e]) {

```

```

12     v = to[e];
13     if (v != f) {
14         w = diameterPath[u] && diameterPath[v] ? -1 : 1;
15         diameter2 = max(diameter2, dist[u] + dist[v] + w);
16         dist[u] = max(dist[u], dist[v] + w);
17     }
18 }
19 }
20
21 int compute() {
22     road(); // 两遍DFS求 diameter1、路径
23     if (k == 1) return 2 * (n - 1) - diameter1 + 1;
24     for (int i = fin; i != 0; i = last[i]) diameterPath[i] = true;
25     memset(dist + 1, 0, n * sizeof(int));
26     dp(1, 0);
27     return 2 * (n - 1) - diameter1 - diameter2 + 2;
28 }

```

题目五：消防 (P2491)

题面

n 个节点、 $n-1$ 条边（边权非负）的树，每个节点是居民点。给定非负整数 s ，可在树上选一条长度不超过 s 的路径建消防站，每个居民去往路径上最近的消防站。求最远居民到消防站的最短距离。

核心思路

- 贪心结论：消防站路径一定建在直径上。
- 两次 DFS 求直径与路径信息；对直径上每个点 i 求 $\text{maxDist}[i]$ = 不经过直径路径上任何点能走出的最大距离（即 i 的侧枝最远距离）。
- 在直径上滑一个长度 $\leq s$ 的窗口（左闭右开 $[l, r)$ ），用单调队列维护窗口内 maxDist 的最大值。每个窗口的答案是三部分取最大：
 - 窗口左侧最远点到窗口左端点的距离 suml ；
 - 窗口内各点 maxDist 的最大值（单调队列队头）；
 - 窗口右侧最远点到窗口右端点的距离 $\text{diameter} - \text{sumr}$ 。
- 所有窗口答案取最小值返回。

坑点

- 必须会单调队列维护窗口内最大值（讲解054），否则窗口滑动时无法 $O(1)$ 得到 maxDist 最大值。
- 窗口是左闭右开 $[l, r)$ ： r 是"右边界再下一个点"， $\text{sumr} - \text{suml}$ 是当前窗口长度，再加 $\text{pred}[r]$ 判断能否继续扩。
- 实际代码是从 end 往 start 跳（因为只有 last 数组能往回跳），与图解方向相反，思路无区别。
- $\text{pred}[i]$ 表示直径上 i 与前一个点之间的边长（以 start 为根），用于计算窗口长度。
- 两遍 DFS 的递归会爆栈，要改成迭代； $\text{maxDistanceExceptDiameter}$ 递归也能过，不用改。

```

1  const int MAXN = 300001;
2  int n, s, start, fin, diameter;

```

```

3  int head[MAXN], nxt[MAXN << 1], to[MAXN << 1], weight[MAXN << 1];
4  int dist[MAXN], last[MAXN], pred[MAXN], maxDist[MAXN];
5  bool diameterPath[MAXN];
6  int que[MAXN];    // 单调队列维护窗口内maxDist最大值
7
8  int maxDistanceExceptDiameter(int u, int f, int c) {
9      int ans = c;
10     for (int e = head[u], v; e != 0; e = nxt[e]) {
11         v = to[e];
12         if (!diameterPath[v] && v != f)
13             ans = max(ans, maxDistanceExceptDiameter(v, u, c + weight[e]));
14     }
15     return ans;
16 }
17
18 int compute() {
19     int suml = 0, sumr = 0, h = 0, t = 0, ans = INT_MAX;
20     // 窗口[l, r), 左闭右开; 从end往start跳
21     for (int l = fin, r = fin; l != 0; l = last[l]) {
22         while (r != 0 && sumr - suml + pred[r] <= s) {
23             while (h < t && maxDist[que[t - 1]] <= maxDist[r]) t--;
24             sumr += pred[r];
25             que[t++] = r;
26             r = last[r];
27         }
28         ans = min(ans, max(max(suml, maxDist[que[h]]), diameter - sumr));
29         if (que[h] == l) h++;
30         suml += pred[l];
31     }
32     return ans;
33 }

```